ÉCOLE NATIONALE SUPERIÉURE D'ARTS ET METIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

PyGym

Application de gestion d'une salle de sport

Projet du module Python avancé

Réalisé par :-**DHIBA YASSINE**
-BAITI ABDERAHMANE
-MOHAMED LOTFANI

Encadré par **: - Prof HAIN**

Code source : **https://github.com/abderrahmanebaiti-star/PyGym-OOP-Project**

1. Présentation de l'application

PyGym est une application de bureau pour une salle de sport. Elle est écrite en Python, avec une interface faite avec Tkinter et une base de données SQLite.

Le membre ouvre l'application et crée un compte avec un nom d'utilisateur et un mot de passe. Après la connexion, il choisit un abonnement. Ensuite il peut réserver des séances (musculation, cardio, yoga, etc.). Chaque abonnement donne accès à certaines activités seulement : plus l'abonnement est cher, plus il y a d'activités.

Les outils utilisés :

- Python : le langage du projet.
- Tkinter : la bibliothèque pour faire les fenêtres, les boutons et les champs. Nous avons aussi utilisé CustomTkinter pour avoir un style plus moderne.
- SQLite : la base de données. Elle est dans un seul fichier (PyGymDB.db), donc il n'y a rien à installer.
- hashlib : pour protéger les mots de passe.
- PyCharm : pour écrire le code.
- GitHub : pour partager le code entre nous trois.

Ce que fait l'application :

L'utilisateur passe par trois écrans, dans cet ordre : l'identification (inscription ou connexion), les abonnements, puis les séances. À chaque écran, deux boutons en haut à gauche (Back et Home) permettent de revenir en arrière ou à l'accueil.

2. Organisation du projet

Le projet est séparé en plusieurs fichiers Python, qui s'appellent entre eux comme le montre ce schéma :

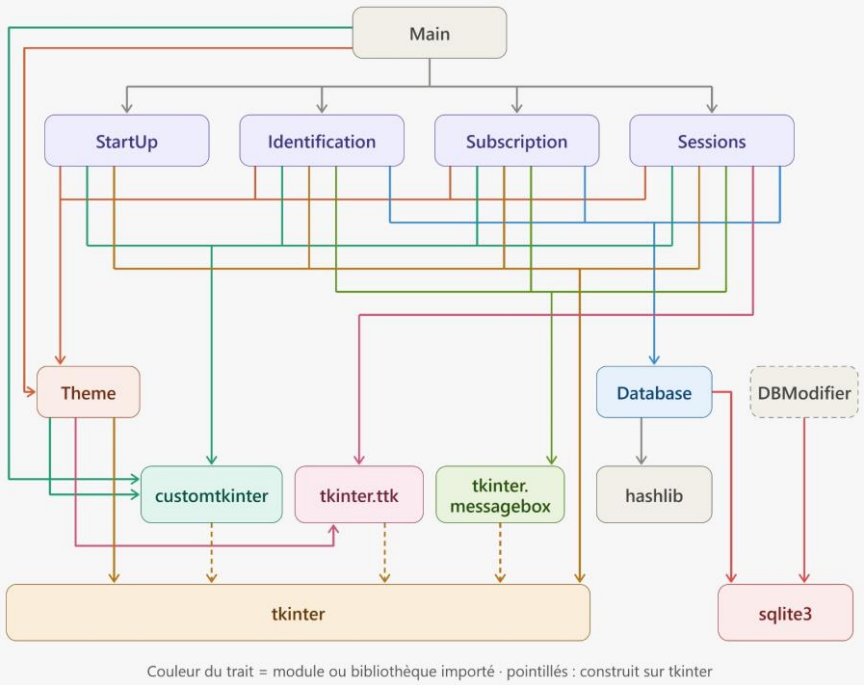


Schéma 1 : les fichiers du projet et ce qu'ils importent

Main lance l'application et relie les quatre écrans : StartUp (accueil), Identification (inscription et connexion), Subscription (abonnements) et Sessions (séances). Tous utilisent Theme pour le style ; Identification, Subscription et Sessions utilisent Database pour les données.

Le fichier principal crée la fenêtre, puis lance l'écran d'accueil. Voici la partie qui décide quel écran s'ouvre après quel autre :

MainPyGym.py (lignes 15 à 29)

```
15  def go_sessions(username):
16      sessions_page(main_window, username, go_subscription)
17
18  def go_subscription(username):
19      Subscription(main_window, username, go_sessions)
20
21  if __name__ == '__main__':
22      try:
23          start(main_window,
24                lambda w: LogIn(w, go_subscription),
25                lambda w: SignUp(w, go_subscription))
26
27          main_window.mainloop()
28      finally:
29          close()
```

Code 1 : le fichier MainPyGym.py

La fonction go_subscription ouvre l'écran des abonnements, et go_sessions ouvre celui des séances. Quand la connexion réussit, on appelle go_subscription. Chaque écran reçoit en paramètre la fonction de l'écran suivant, donc les fichiers ne dépendent pas les uns des autres. Le bloc finally ferme la base de données quand on quitte l'application.

3. La base de données

La base de données est faite avec SQLite et contient 3 tables :

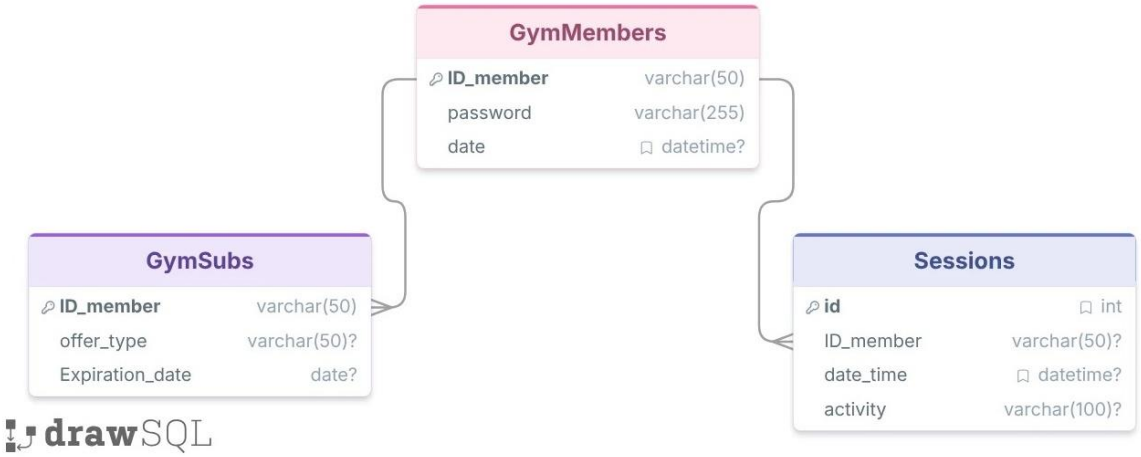


Schéma 2 : le modèle de la base de données (3 tables liées par ID_member)

GymMembers contient les membres (ID_member est le nom d'utilisateur et date la date d'inscription), GymSubs l'abonnement d'un membre (l'offre et sa date de fin) et Sessions les séances réservées.

Les tables sont liées par la colonne ID_member. Un membre n'a qu'un seul abonnement, parce que ID_member est la clé primaire de GymSubs. Par contre, il peut avoir beaucoup de séances.

Chaque fois que l'application ouvre la base, elle supprime les abonnements finis avec la requête DELETE FROM GymSubs WHERE CURRENT_DATE > Expiration_date. Comme ça, un membre dont l'abonnement est terminé ne peut plus réserver de séance.

DatabasePyGym.py (lignes 21 à 40)

```
21  PyGymDB = get_connection()
22  # Initialisation Identification
23  PyGymDB.execute('CREATE TABLE IF NOT EXISTS GymMembers ('
24                  'ID_member VARCHAR PRIMARY KEY,'
25                  'password VARCHAR,'
26                  'date TIMESTAMP)')
27  # Initialisation Subscription
28  PyGymDB.execute('CREATE TABLE IF NOT EXISTS GymSubs ('
29                  'ID_member VARCHAR PRIMARY KEY,'
30                  'offer_type VARCHAR,'
31                  'Expiration_date DATE,'
32                  'FOREIGN KEY (ID_member) REFERENCES GymMembers(ID_member))')
33  # Initialisation Session
34  PyGymDB.execute('CREATE TABLE IF NOT EXISTS Sessions ('
35                  'id INTEGER PRIMARY KEY AUTOINCREMENT,'
36                  'ID_member VARCHAR,'
37                  'date_time TEXT,'
38                  'activity VARCHAR,'
39                  'FOREIGN KEY (ID_member) REFERENCES GymMembers(ID_member))')
40  PyGymDB.commit()
```

Code 2 : création des 3 tables (DatabasePyGym.py)

CREATE TABLE IF NOT EXISTS crée la table seulement si elle n'existe pas déjà, donc on peut lancer le programme plusieurs fois. PRIMARY KEY veut dire que la valeur est unique pour chaque ligne. FOREIGN KEY relie ID_member à la table GymMembers. À la fin, commit() enregistre les changements.

4. Les comptes des membres

Les mots de passe ne sont jamais gardés en clair dans la base. La fonction hash_password les transforme avec sha256 avant de les enregistrer.

DatabasePyGym.py (lignes 1 à 5)

```
1  from sqlite3 import connect, OperationalError
2  from hashlib import sha256
3
4  def hash_password(password):
5      return sha256(password.encode()).hexdigest()
```

À la connexion, on transforme aussi le mot de passe écrit par l'utilisateur, puis on compare les deux résultats. Si quelqu'un ouvre la base, il ne voit pas les vrais mots de passe.

Deux fonctions s'occupent des membres dans la base :

DatabasePyGym.py (lignes 43 à 55)

```
43 # Partie Identification:
44 def get_password(username):
45     row = PyGymDB.execute('SELECT password '
46                           'FROM GymMembers '
47                           'WHERE ID_member = ?', (username,)).fetchone()
48     return row[0] if row else None
49
50
51 def add_member(username, password):
52     PyGymDB.execute('INSERT INTO GymMembers(ID_member,password,date) '
53                     'VALUES (?, ?, CURRENT_DATE)',
54                     (username, password))
55     PyGymDB.commit()
```

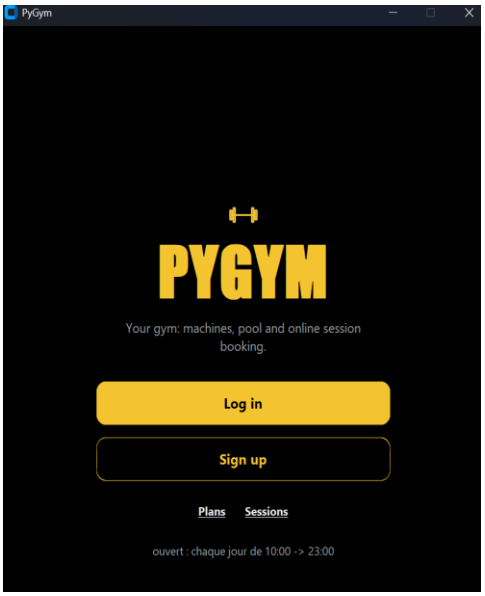
Code 4 : les fonctions get_password et add_member

get_password cherche le mot de passe d'un membre. Le point d'interrogation est remplacé par la valeur de username. Cette méthode évite les erreurs et les attaques SQL. fetchone() donne une ligne, ou None si le membre n'existe pas.

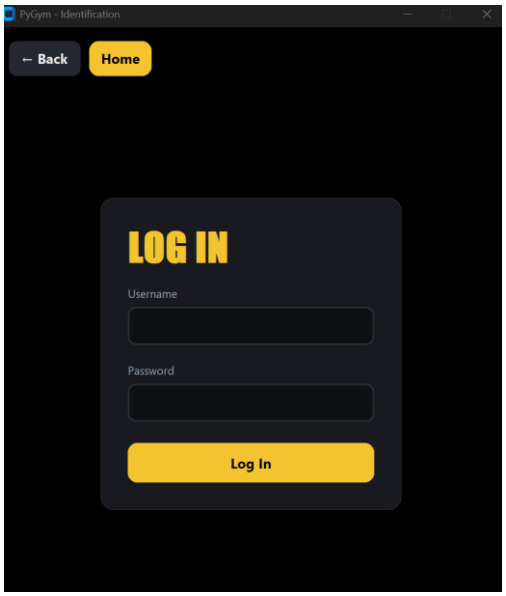
add_member ajoute un nouveau membre avec la date du jour (CURRENT_DATE).

5. Inscription et connexion

L'écran d'accueil propose deux boutons : Log in et Sign up. Dans les deux cas, l'utilisateur écrit son nom d'utilisateur et son mot de passe.



Écran d'accueil



Écran de connexion (Log In)

Voici la fonction login, qui vérifie la connexion :

IdentificationPyGym.py (lignes 55 à 70)

```
55      def login(self):
56          input_username = self.username.get().strip()
57          correct_password = get_password(input_username)
58          if correct_password is not None:
59              if correct_password == hash_password(self.password.get()):
60                  showinfo(message="Logged in successfully.")
61                  self.endID()
62                  self.for_next(input_username)
63              else:
64                  try_again = askretrycancel(title="Error",
65                                             message="Incorrect password. Try Again?")
66                  if try_again:
67                      self.password.set('')
68                  else:
69                      self.window.destroy()
70                      close()
```

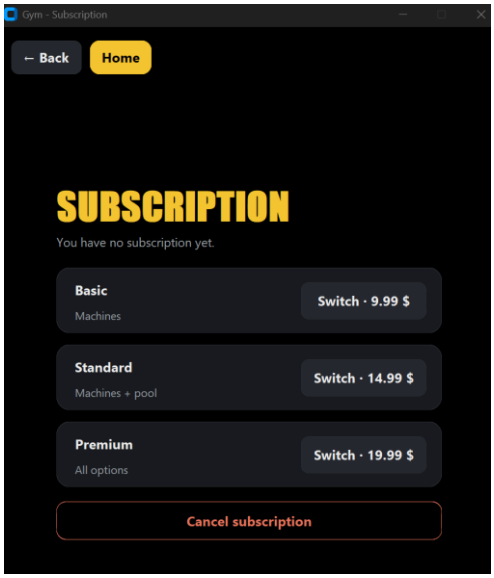
Code 5 : la fonction login (début)

La fonction lit le nom d'utilisateur, puis cherche son mot de passe dans la base. Si le mot de passe est bon, un message s'affiche et on ouvre l'écran suivant avec self.for_next. S'il est faux, un message propose de réessayer. Si le compte n'existe pas (la suite de la fonction), l'application propose de créer un compte.

6. Les abonnements

Il y a trois offres. Chaque abonnement dure 30 jours.

Offre	Prix	Activités permises
Basic	9.99 $	Musculation, Cardio
Standard	14.99 $	Basic + Swimming, Aquagym
Premium	19.99 $	Standard + Yoga, CrossFit, Box



Écran Subscription

Dans cet écran, le membre voit les trois offres. L'offre qu'il a déjà est marquée ACTIVE. Il peut passer à une offre plus chère avec le bouton Switch.

Le dernier bouton permet de passer à l'écran des séances sans rien acheter.

DatabasePyGym.py (lignes 64 à 74)

```
64  def add_subscription(username, offer):
65      if get_subscription(username) is None:
66          PyGymDB.execute('INSERT INTO GymSubs (ID_member, offer_type, Expiration_date) '
67                          "VALUES (?, ?, date('now', 'localtime', ?))",
68                          (username, offer, f"+{valid_time} days"))
69      else:
70          PyGymDB.execute('UPDATE GymSubs '
71                          "SET offer_type = ?, Expiration_date = date('now', 'localtime', ?) "
72                          'WHERE ID_member = ?',
73                          (offer, f"+{valid_time} days", username))
74      PyGymDB.commit()
```

Code 6 : la fonction add_subscription

Si le membre n'a pas encore d'abonnement, on fait un INSERT. S'il en a déjà un, on fait un UPDATE pour changer l'offre. La date de fin est calculée par SQLite : date('now', 'localtime', '+30 days') veut dire aujourd'hui plus 30 jours.

7. Les séances : les règles

Chaque activité demande un niveau d'abonnement minimum. Nous avons mis ces niveaux dans deux dictionnaires :

DatabasePyGym.py (lignes 78 à 90)

```
78  offer_level = {"Basic": 1,
79                 "Standard": 2,
80                 "Premium": 3}
81
82  activities = {
83      "Musculation": 'Basic'    ,          # machines
84      "Cardio"     : 'Basic'    ,          # machines
85      "Swimming"   : 'Standard',          # machines + pool
86      "Aquagym"    : 'Standard',          # machines + pool
87      "Yoga"       : 'Premium' ,          # all options
88      "CrossFit"   : 'Premium' ,
89      "Box"        : 'Premium' ,
90  }
```

Code 7 : les niveaux et les activités (DatabasePyGym.py)

Par exemple, le Yoga demande Premium, donc le niveau 3. Un membre Basic (niveau 1) ne peut pas le réserver.

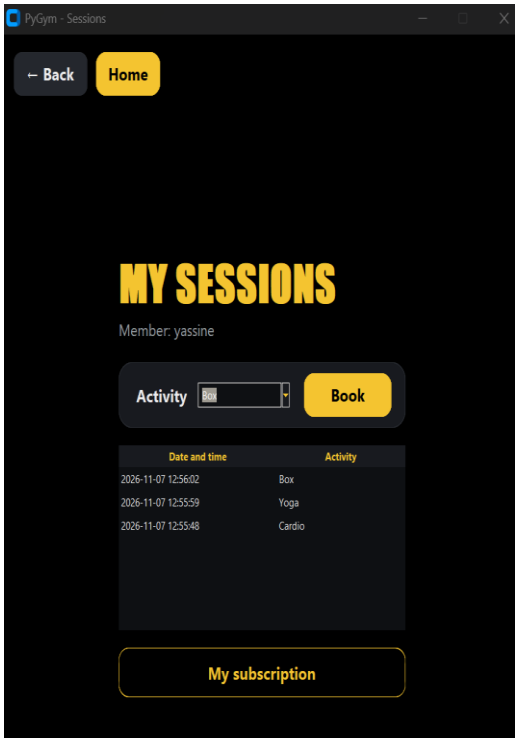
DatabasePyGym.py (lignes 92 à 105)

```
92  def allowed_activities(username):
93      offer = get_subscription(username)
94      if offer is None:
95          return []
96      return [name for name, required in activities.items() if offer_level[offer] >= offer_level[required]]
97
98  def add_session(username, activity):
99      if activity in allowed_activities(username):
100         PyGymDB.execute('INSERT INTO Sessions(ID_member, date_time, activity) '
101                         "VALUES (?, datetime('now','localtime'), ?)",
102                         (username, activity))
103         PyGymDB.commit()
104         return True
105     return False
```

Code 8 : les fonctions allowed_activities et add_session

allowed_activities compare le niveau de l'abonnement du membre avec le niveau de chaque activité. Si le membre n'a pas d'abonnement, elle renvoie une liste vide. add_session enregistre la séance seulement si l'activité est dans cette liste. Elle renvoie True si c'est bon et False sinon.

8. L'écran des séances



Écran My sessions

Dans cet écran, le membre choisit une activité dans une liste, puis clique sur le bouton Book. La liste montre seulement les activités permises par son abonnement.

Le tableau en bas affiche l'historique de ses séances, avec la date, l'heure et l'activité.

Voici la fonction du bouton Book :

SessionsPyGym.py (lignes 34 à 41)

```
34      def book():
35          if add_session(username, activity.get()):
36              showinfo("Session",
37                       "Session booked!")
38              fill_table()
39          else:
40              showerror("Refused",
41                        "You need a higher subscription to book this session.")
```

Code 9 : la fonction book (SessionsPyGym.py)

book appelle add_session. Si elle renvoie True, un message confirme la réservation et le tableau est mis à jour avec fill_table. Sinon, un message d'erreur dit qu'il faut un abonnement plus haut pour cette séance.

9. Le thème graphique

Le fichier ThemePyGym.py regroupe tout le style de l'application au même endroit. Tous les écrans l'importent (import ThemePyGym as theme), donc un changement dans ce fichier change l'apparence de toute l'application.

Les couleurs sont des variables : noir pour le fond (bg), jaune pour les accents (yellow), gris pour les textes secondaires (muted) et rouge pour les actions dangereuses (red). Les styles CARD, PRIMARY, OUTLINE, DARK et DANGER sont des dictionnaires de paramètres (dict).

ThemePyGym.py (lignes 5 à 6)

```
5  bg, card, border, field = "#000000", "#181A1F", "#2A2D33", "#0E1013"
6  yellow, yellow_hover, text, muted, red = "#F4C430", "#FFD95A", "#EDEDED", "#9AA0A6", "#E8735A"
```

Code 10 : les couleurs (ThemePyGym.py)

La fonction set_background choisit les polices (Bebas Neue et Inter si elles sont installées, sinon Impact et Segoe UI), puis règle le fond de la fenêtre, le logo et le style du tableau et de la liste des activités.

ThemePyGym.py (lignes 24 à 27)

```
24      title_font = "Bebas Neue" if "Bebas Neue" in font.families() else "Impact"
25      body_font = "Inter"       if "Inter"      in font.families() else "Segoe UI"
26      TITLE_BIG, TITLE, PRICE = (title_font, 76, "bold"), (title_font, 46, "bold"), (title_font, 32, "bold")
27      BOLD, TEXT, SMALL =       (body_font, 16, "bold") , (body_font, 15)         , (body_font, 13)
```

Code 11 : le choix des polices (ThemePyGym.py)

La fonction nav_bar affiche les boutons Back et Home sur chaque écran. Elle garde les écrans visités dans la liste history : Back rouvre l'écran précédent et Home revient à l'accueil.

10. Conclusion

PyGym permet de créer un compte, de choisir un abonnement (Basic, Standard ou Premium) et de réserver des séances selon son offre. Ce projet nous a permis de combiner Python (un code séparé en plusieurs fichiers), SQLite (3 tables liées) et Tkinter avec CustomTkinter (des écrans qui partagent un même thème).